Demo Script (5 minutes)

Prepare before the demo (not counted): venv active (`source .venv/bin/activate`), a test file, four terminals, Wireshark open on **Loopback: lo0** with `mpft.lua` installed, a pre-captured screenshot set as backup.

```
mkdir -p /tmp/demo && head -c 60000000 /dev/urandom > /tmp/demo/video.bin   # 60 MB
cat > /tmp/demo/topo.json <<'J'
[{"listen_port":9201,"to":"127.0.0.1:9000","delay_ms":5, "rate_mbit":40},
 {"listen_port":9202,"to":"127.0.0.1:9000","delay_ms":20,"rate_mbit":20},
 {"listen_port":9203,"to":"127.0.0.1:9000","delay_ms":20,"rate_mbit":20,"drop_after_s":6}]
J
```

0:00 - 1:00 Problem and topology

Say: one TCP connection uses one route, stalls when it breaks, and a corrupted or half-sent file is wasted. MPFT splits the file into chunks, sends them over several TCP paths, verifies SHA-256, and resumes. Show `docs/TOPOLOGY.md` ASCII diagram: sender, 3 emulated paths (fast, medium, one that dies at 6 s), receiver.

1:00 - 4:00 Live transfer with a path killed

Terminal 1 (receiver):

```
python -m mpft receive --listen 127.0.0.1:9000 --out /tmp/demo/out
```

Terminal 2 (emulator, 3 paths, third dies 6 s after its first connection):

```
python -m mpft emulate --config /tmp/demo/topo.json
```

Terminal 3 (dashboard; open http://127.0.0.1:8080):

```
python -m mpft dashboard
```

Terminal 4 (sender, start Wireshark capture first):

```
python -m mpft -v send /tmp/demo/video.bin --to 127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203 --chunk-size 256k
```

Point at: three bars on the dashboard, faster path carries more, at about 6 s the third path turns `down`, bars of the others stay up, log line "path 2 failed ... requeueing", transfer still ends with `OK`. Then prove integrity:

```
shasum -a 256 /tmp/demo/video.bin /tmp/demo/out/video.bin
```

Optional corruption run (shows NACK in the receiver log):

```
python -m mpft send /tmp/demo/video.bin --to 127.0.0.1:9201 --paths 2 --corrupt-rate 0.05
```

Note: the emulator's third path is already dead after the first run; restart the emulator (Ctrl-C, run again) before repeating.

4:00 - 5:00 Wireshark, then results

Wireshark (about 30 s): filter `mpft.type == 2` (decoded MANIFEST JSON), `mpft.type == 4` (CHUNK header fields), `tcp.flags.syn == 1` (one handshake per path), Statistics > Conversations (bytes per path). Results (about 30 s): open `results/throughput_vs_paths.png` (goodput doubles with paths) and `results/recovery_timeline.png` (one path stops, others carry on, recovery 0.15 s).

Web UI demo (about 4 minutes, one Mac)

1. Start rendezvous, alice (8000/8001) and bob (8010/8011) as in WEB.md; open bob's Receive page in tab 1, alice's Send page in tab 2.
2. Alice: choose a 100 MB file, paste bob's ID, Advanced > Network conditions = Asymmetric, Send. Bob: press Accept (point out that a code would skip this click).
3. Alice's Transfer page: chunk map filling, four paths at different Mbit/s, chart with one line per path.
4. While it runs, press Kill path 1 under Network conditions. Show the event log (path down, chunks requeued) and that the other paths finish the file.
5. Set corruption to 5 %: show "chunk N hash mismatch, retransmitting" and the Chunks failed counter.
6. Show Verification: expected and computed SHA-256 equal, "Hash verified". On bob's Receive page, Download the file and run `shasum -a 256`.
7. Copy the tcpdump command from the Wireshark panel for the next run, then filter `mpft.type == 4`.

Two-device web demo (about 5 minutes, two Macs over Tailscale)

1. Both Macs run `python -m mpft web --name NAME --ui-port 8000 --peer-port 8001 --data-port 9000 --out ./downloads`. Receiver shows Receive > Copy ID and sends it to the sender.
2. Sender: Send, pick a 200 MB file, paste the ID, Test connection (reachable), Advanced > Network conditions = Lossy path, Send. Receiver presses Accept.
3. Both screens show the same Transfer page: chunk map, paths table, graphs. Point at the receiver's screen showing the sender's events.
4. Sender kills path 1 in the Network panel. Show the incident marker "path 1 down" on the graphs, the Incidents row ("returned to the queue; path 0 delivered them") and the recovery time; press Restore and watch the path come back.
5. Set corruption to 5 %: Verification shows rejected chunk indexes. Finish: "Hash verified"; receiver Downloads and runs `shasum -a 256`.
6. History > Replay the transfer at 16x. Then show the Wireshark panel command and filter `mpft.type == 4` on the capture.

Rehearsal checklist

- [ ] Time each section with a stopwatch; cut the corruption run first if over time
- [ ] Ports 9000, 9201-9203, 8080 are free (`lsof -i :9201`)
- [ ] Web demo: ports 8000, 8001, 8010, 8011, 9100 free; latest `wireshark/mpft.lua` copied into Wireshark
- [ ] Two-device demo: both Macs on Tailscale, `--data-port 9000` allowed through the macOS firewall, Settings > self-test green on both
- [ ] Backup: screenshots of dashboard and Wireshark if the live demo fails

Scenario tour (about 5 minutes)

1. Send a 30 MB file with **Clean but slow**: the sender page says "Sending" and "Delivered"; open Bob's page: "Receiving", verification first, Download button.
2. Send with **Path flaps**: the incident timeline shows path down, backoff, reconnect, recovered.
3. Send with **Heavy loss**: rejected chunks and retransmissions, file hash still matches.
4. Send an 8 MB file with **Total failure**: sender "Not delivered", receiver "Nothing was saved", no Download.
5. Send a 2 MB file on a clean network: note the graphs use milliseconds.